GPS for Career Upskilling

1. Introduction

This project develops an end-to-end analytics and recommendation system titled “GPS for Career Upskilling”, which aims to generate personalized learning roadmaps based on a user’s current skillset, target job requirements, and time constraints. The system addresses a critical challenge in modern online education: the overwhelming abundance of learning resources and the absence of structured, personalized roadmaps. As highlighted in the project concept, users often experience “learning path paralysis,” spending significant time identifying relevant courses without a clear strategy for progression.

The primary objective of this system is to transform unstructured textual data—namely resumes and job descriptions—into actionable insights. Specifically, the system extracts skills from both sources, identifies gaps between current and desired competencies, and recommends an optimized sequence of learning resources. This is achieved through the integration of natural language processing (NLP) techniques and an optimization framework that maximizes learning utility under real-world constraints.

The overall architecture consists of three interconnected modules: (1) skill extraction from resumes and job descriptions, (2) gap analysis and skill scoring, and (3) an optimization engine for course selection. As illustrated in the system design, the final output is an optimized learning roadmap that ranks skills by priority and selects courses that best address the user’s skill deficiencies within a limited time budget. This integrated approach enables a data-driven and scalable solution to personalized career development.

2. Methodology and Models

2.1 Resume Skill Extraction using SkillNER

The first stage of the system involves extracting structured skill information from unstructured resume text. This is achieved using a pre-trained SkillNER model, which is based on Named Entity Recognition (NER). The model is designed to identify spans of text corresponding to skills, treating them as named entities within the document.

The extraction pipeline consists of several preprocessing and modeling steps. Initially, raw resume text is cleaned by removing irrelevant symbols, punctuation, and noise. This is followed by tokenization and linguistic normalization, including lemmatization and part-of-speech tagging. The SkillNER model then applies a combination of statistical and rule-based matching techniques to identify skill entities. These include full matching, n-gram matching, and token-level similarity scoring, which allow the model to capture both exact matches and approximate variations of skill expressions.

The output of this stage is a list of extracted skills for each resume, which serves as the foundation for subsequent analysis. Importantly, this approach allows the system to convert highly unstructured textual data into a structured representation suitable for quantitative modeling.

To assess the performance of the SkillNER model, its outputs were compared against an alternative extraction method based on an external API. The evaluation metric used is the Jaccard similarity coefficient, defined as:

$$\text{Accuracy} = \frac{|S_{NER} \cap S_{API}|}{|S_{NER} \cup S_{API}|}$$

where S_{NER} represents the set of skills extracted by SkillNER and S_{API} represents the set extracted by API. This metric measures the proportion of shared skills relative to the total unique skills identified by both methods.

Empirical results indicate that the average similarity score is approximately 10.02%, with individual examples such as 16.19% overlap. At first glance, these results suggest poor agreement between the two methods. However, a deeper analysis reveals that the low similarity is primarily driven by differences in skill representation rather than true extraction errors. For example, variations such as “Excel” versus “Microsoft Excel” or “ML” versus “Machine Learning” are treated as distinct entries, thereby reducing the intersection size while inflating the union.

Furthermore, the API-based method tends to produce a larger and noisier set of skills, which increases the denominator in the Jaccard calculation and artificially lowers the score. Consequently, while the numerical evaluation suggests low performance, manual inspection indicates that the SkillNER model captures relevant skills with reasonable accuracy. This highlights the limitations of using set-based similarity metrics in contexts where semantic equivalence is not explicitly accounted for.

2.2 Job Description Skill Extraction: Dual-Engine Model

In contrast to resume processing, skill extraction from job descriptions employs a dual-engine architecture designed to balance precision and recall. This model consists of two complementary components: a dictionary-based matcher and a custom machine learning NER model.

The first component, the dictionary matcher, relies on a curated taxonomy of skills comprising 47 canonical skills and 446 alias entries. This approach ensures high precision by matching known skill terms and their variations. However, it is limited in its ability to capture unseen or domain-specific skills that are not included in the predefined dictionary.

To address this limitation, the second component—a custom ML-based NER model—is introduced. This model is capable of identifying out-of-vocabulary (OOV) skills by learning contextual patterns in the text. While this component improves recall, it may introduce additional noise due to less constrained extraction.

The outputs of both engines are subsequently merged and deduplicated. A prioritization rule is applied in which dictionary-based matches take precedence, while the ML model is used to supplement missing skills. This hybrid approach ensures a balance between accuracy and coverage, enabling the system to extract a comprehensive set of skills from job descriptions.

2.3 Skill Scoring

Following skill extraction, each skill identified in the job description is assigned a relevance and difficulty score. These scores quantify the importance and complexity of each skill, forming the basis for prioritization.

The relevance score is computed as:

$$\text{Relevance} = \left(TF \cdot w_{tf} + Position \cdot w_p + Signal \cdot w_s \right) \times 100$$

where:

- TF (term frequency) is defined as: $TF = \dfrac{\log(1 + \text{mentions})}{\log(1 + \text{total sentences})}$
- $Position$ represents the proportion of mentions occurring in critical sections: $Position = \dfrac{\text{mentions in requirements}}{\text{total mentions}}$
- $Signal$ captures the linguistic strength of the surrounding context, calculated as the mean importance weight of sentences containing the skill.

This formulation ensures that skills mentioned frequently, emphasized in important sections, and associated with strong linguistic signals (e.g., “must have”) receive higher scores.

The difficulty score is derived from textual modifiers associated with each skill. Words such as “expert,” “advanced,” or “basic” are mapped to numerical values within a predefined scale ranging from 0 to 100. This allows the model to quantify the expected proficiency level required for each skill.

The final priority score is computed as a weighted combination of relevance and difficulty:

$$\text{Priority} = 0.65 \cdot \text{Relevance} + 0.35 \cdot \text{Difficulty}$$

This prioritization mechanism enables the system to rank skills according to both their importance in the job description and their level of difficulty, providing a structured basis for gap analysis.

2.4 Optimization Model for Course Recommendation

The final stage of the system involves recommending courses that address the identified skill gaps. Each course is assigned a utility score based on multiple factors:

$$\text{CourseUtility}_i = w_1 \cdot \text{SkillCoverage}_i + w_2 \cdot \text{DifficultyScore}_i + w_3 \cdot \text{QualityScore}_i - w_4 \cdot \text{TimePenalty}_i$$

where:

- SkillCoverage measures how well the course covers missing skills,
- DifficultyScore reflects alignment with the user’s proficiency level,
- QualityScore is derived from ratings and reviews,
- TimePenalty is defined as: $\text{TimePenalty}_i = \dfrac{\text{course hours}_i}{\text{time budget}}$

The optimization problem is formulated as:

$$\max \sum_{i=1}^{N} \text{CourseUtility}_i \cdot x_i$$

$$\text{s.t.} \sum_{i=1}^{N} \text{CourseTime}_i \cdot x_i \leq \text{TimeBudget}$$

$$x_i \in \{0,1\}$$

Due to computational complexity, a greedy algorithm is employed to approximate the solution. At each iteration, the course with the highest utility score is selected, the covered skills are removed from the gap list, and the remaining utilities are recalculated. This process continues until the time constraint is reached. While this approach does not guarantee a global optimum, it provides a computationally efficient and practically effective solution.

3. Findings and Outcomes

The implementation of the proposed system demonstrates the feasibility of integrating NLP and optimization techniques for personalized learning recommendations. The SkillNER model successfully extracts structured skill information from resumes, enabling effective comparison with job requirements. Although the quantitative evaluation using Jaccard similarity yields low scores, qualitative analysis suggests that the model captures relevant skills with reasonable accuracy.

The dual-engine model for job description processing proves to be particularly effective, combining the strengths of rule-based and machine learning approaches. By leveraging both high-precision dictionary matching and high-recall NER extraction, the system achieves a balanced and comprehensive skill representation.

The skill scoring framework further enhances the system by introducing a structured method for prioritizing skills. The combination of relevance and difficulty scores allows for nuanced ranking, ensuring that critical and attainable skills are emphasized in the learning roadmap.

Finally, the optimization model successfully generates personalized course recommendations under time constraints. By formulating the problem as a constrained utility maximization task, the system ensures that selected courses provide maximum value relative to the user’s goals and limitations. The greedy algorithm, while approximate, delivers efficient and interpretable solutions suitable for real-time applications.

4. Conclusion

The "GPS for Career Upskilling" project successfully demonstrates a scalable, data-driven framework to resolve "learning path paralysis" by bridging the gap between a user’s current capabilities and market demands. By integrating Natural Language Processing (NLP) with operations research methodologies, the system translates highly unstructured text from resumes and job descriptions into actionable, personalized learning roadmaps.

Key successes of the system include:

- **Structured Knowledge Extraction:** The deployment of the SkillNER model for resumes and a hybrid Dual-Engine model for job descriptions successfully captures both standard and out-of-vocabulary (OOV) skills, balancing high precision with strong recall.

- **Nuanced Gap Analysis:** Moving beyond simple keyword matching, the system successfully quantifies skill relevance and difficulty. This ensures that the generated roadmaps prioritize competencies based on their actual weight and context within a target job.

- **Practical Recommendation Engine:** By framing course selection as a constrained utility maximization problem, the system effectively mimics real-world limitations. The application of a greedy algorithm provides computationally efficient, highly relevant course recommendations that respect a user's time constraints.

Ultimately, this project proves the viability of using automated analytics to navigate the overwhelming abundance of online education, providing users with a strategic, prioritized path toward their career goals.

5. Future Improvements

While the current architecture provides a robust foundation for personalized upskilling, several enhancements could further optimize the system's accuracy and utility:

- **Semantic Matching Over Lexical Matching:** As highlighted by the artificially low Jaccard similarity scores, exact string matching fails to capture semantic equivalence (e.g., "ML" vs. "Machine Learning"). Future iterations should integrate dense vector embeddings (such as BERT or domain-specific sentence transformers) to group aliases and conceptually related skills, yielding a more accurate gap analysis.

- **Advanced Optimization Algorithms:** The current course recommendation engine relies on a greedy algorithm. While computationally efficient, it does not guarantee a global optimum. Implementing exact solvers using Integer Linear Programming (ILP) or heuristic approaches like Genetic Algorithms could yield better course combinations, particularly for users with larger time budgets or complex prerequisite structures.

- **Dynamic Skill Taxonomy Generation:** The dictionary-based matcher relies on a static taxonomy of 47 canonical skills and 446 aliases. Integrating automated ontology generation —mining current job boards periodically to identify and append trending skills—would prevent the taxonomy from becoming obsolete in rapidly evolving fields like technology and data science.

- **Refined Proficiency Assessment:** Currently, skill difficulty and user proficiency are inferred from textual modifiers (e.g., "expert," "basic"). Incorporating adaptive user assessments or extracting chronological experience data from resumes (e.g., calculating years of experience with a specific tool) would create a more accurate baseline of the user's current capabilities.

Appendix

A. Glossary of Terms

- **NER (Named Entity Recognition):** A sub-task of information extraction that locates and classifies named entities mentioned in unstructured text into predefined categories (in this context, professional skills).

- **SkillNER:** An NLP-based model specifically trained to detect and extract skill entities from textual data.
- **OOV (Out-of-Vocabulary):** Terms or skills that are not present in the system's predefined dictionary or training dataset.
- **Dual-Engine Architecture:** A hybrid extraction methodology combining rule-based dictionary matching (for high precision) and machine learning (for high recall).

B. Mathematical Formulations

The following formulas represent the core mathematical logic driving the system's analytical and recommendation engines:

1. Jaccard Similarity Coefficient Used to evaluate the overlap between different extraction methods:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

(Where A is the set of skills extracted by SkillNER and B is the set extracted by the API.)

2. Relevance Score Formulation A composite score reflecting a skill's importance in a job description:

$$\text{Relevance} = w_1(TF) + w_2(S_{critical}) + w_3(L_{strength})$$

3. Priority Score Combines both the importance of the skill and the level of proficiency required:

$$\text{Priority} = \alpha \times \text{Relevance} + \beta \times \text{Difficulty}$$

4. Utility Score Evaluates the value of a specific course recommendation:

$$U_i = \text{SkillCoverage}_i + \text{DifficultyScore}_i + \text{QualityScore}_i - \text{TimePenalty}_i$$

5. Optimization Problem (Knapsack Formulation) Maximizes the total learning utility without exceeding the user's maximum time budget (T):

$$\max \sum_{i=1}^{n} U_i x_i$$

$$\text{s.t.} \sum_{i=1}^{n} t_i x_i \leq T, \quad x_i \in \{0, 1\}$$